

Damose: Software Design Decision Summary

Date: 31/10/2025

Features Implemented

1. Lettura dei dati GTFS statici: importazione e parsing dei file CSV contenenti informazioni su fermate, linee, viaggi, orari e shape del percorso (tramite la classe *MyParser*). Gestione strutturata dei dati nei model dedicati (*Stop*, *Route*, *Trip*, *Shape*, *StopTime*, *Agency*).
2. Visualizzazione della mappa con fermate: utilizzo di *JXMapView* per rappresentare la mappa geografica. Ogni fermata (*Stop*) è mostrata come un waypoint con icona personalizzata. Possibilità di cliccare sulle fermate per aprire il relativo pannello informativo (*PannelloFermata*).
3. Ricerca fermate e linee: campo di ricerca dinamico nel *PannelloRicerca* con aggiornamento in tempo reale. Ricerca per nome o ID di fermata e per codice/nome di linea. Selezione del risultato per centrare e zoomare la mappa sulla fermata o mostrare il percorso della linea.
4. Visualizzazione percorso linea: tracciamento grafico delle shape di andata e ritorno sulla mappa con colori distinti. Calcolo automatico del fit della vista per visualizzare l'intero percorso.
5. Gestione del pannello laterale dinamico (*MyFrame*): interfaccia con più sezioni (*Ricerca*, *Preferiti*, *Utente*, *Impostazioni*, *Linea*). Pannello laterale che si apre o chiude dinamicamente in base al contesto selezionato.
6. Gestione dei preferiti (*PannelloPreferiti* e *GestorePannelloPreferiti*): possibilità di aggiungere o rimuovere linee/fermate dai preferiti. Salvataggio dei preferiti durante la sessione. Visualizzazione e selezione rapida dalla lista dei preferiti.
7. Gestione utente (*PannelloUtente* e *GestorePannelloUtente*): inserimento (nome, email, password). Simulazione della gestione profilo (work in progress, c'è solo la view).
8. Pannello impostazioni (*PannelloImpostazioni*): accerta la modalità operativa (offline/online). Visualizzazione dello stato attuale del sistema.
9. Gestione centralizzata della mappa (*GestoreMappa*): coordinamento tra waypoint, percorsi e refresh grafico. Renderer personalizzato (*ImageWaypointRenderer*) per

icone di bus e fermate.

10. Gestione modalità online/offline (*GestoreMode*): gestione del comportamento in modalità online, controllo centralizzato per l'attivazione o disattivazione di funzionalità in base alla modalità.
 11. Integrazione dati in tempo reale (*GestoreRealTime*): timer per aggiornamento periodico delle posizioni dei bus e orari d' arrivo . Visualizzazione dinamica dei bus sulla mappa tramite icone sovrapposte.
 12. Interfaccia modulare e estensibile: architettura *MVC* con separazione netta tra *Model* (gestione dei dati GTFS), *View* (interfacce grafiche Swing) e *Controller* (logica applicativa). Design scalabile per aggiungere nuove tipologie di veicoli, pannelli o sorgenti dati.
-

1. Project Overview

Project Name: Damose – Sistema di Visualizzazione e Ricerca GTFS (Roma)

Il progetto Damose è un'applicazione desktop che consente di visualizzare e gestire in modo interattivo i dati GTFS (General Transit Feed Specification).

Il software permette di cercare linee e fermate, visualizzarle su una mappa, gestire preferiti ed ha la modalità online/offline con aggiornamenti real-time.

L'obiettivo è fornire un sistema modulare e scalabile per la consultazione di linee di trasporto pubblico, separando chiaramente la logica dei controller, l'interfaccia grafica e la gestione dei dati.

2. Object-Oriented Design (OOD) Decisions

2.1 Class Design & Responsibilities

- **Class 1: MyFrame**
Primary Responsibility: Gestione della finestra principale dell'applicazione (GUI principale).
Justification: Centralizza la visualizzazione dei pannelli (ricerca, preferiti, impostazioni, utente). Gestisce la visibilità e il layout della finestra principale.
- **Class 2: MappaAutobus**
Primary Responsibility: Visualizzazione grafica su mappa delle fermate e linee GTFS.
Justification: Incapsula tutte le operazioni legate a JXMapView, waypoint e painters. Consente la rappresentazione geografica delle entità del modello.
- **Class 3: GestorePannelloRicerca**
Primary Responsibility: Gestione logica del pannello di ricerca (filtri, interazioni, aggiornamenti della mappa).
Justification: Implementa la logica dell'interfaccia ricerca, gestendo le interazioni utente e il filtraggio.
- **Class 4: MyParser**
Primary Responsibility: Lettura e parsing dei file GTFS (.txt) per popolare i modelli (Stop, Route, Trip, Shape, ecc.).
Justification: Separa la logica di input dei dati dal resto del programma, rispettando il principio di separazione delle responsabilità (SRP).
- **Class 5: Stop / Route / Trip / Shape / StopTime / Agency / CalendarDate / BusWaypoint**
Primary Responsibility: Modelli di dati che rappresentano le entità GTFS.
Justification: Ogni classe incapsula i dati relativi a un concetto reale (fermata, linea, viaggio, ecc.), rispettando i principi di modellazione OOP.

2.2 Encapsulation

Tutti gli attributi delle classi di modello e vista sono dichiarati private, garantendo l'incapsulamento.

L'accesso è controllato tramite metodi getter e setter pubblici, esposti solo dove necessario. Alcuni campi vengono gestiti con Reflection (ad esempio *pannelloVisibile* in *MyFrame*) per resettare dinamicamente lo stato interno senza esporre metodi pubblici.

Benefici:

- Protezione dell'integrità dei dati
 - Riduzione dell'accoppiamento tra le classi
 - Migliore manutenibilità e leggibilità del codice
-

2.3 Inheritance

Il progetto utilizza composizione più che ereditarietà, coerentemente con l'architettura MVC. Tuttavia, alcune classi Swing (JPanel, JFrame) sono estese per costruire componenti personalizzati dell'interfaccia grafica (es. *MyFrame*, *PannelloRicerca*, *PannelloPreferiti*).

- Base Class: JPanel, JFrame
- Derived Classes: PannelloRicerca, PannelloUtente, PannelloImpostazioni, ecc.

Motivazione: L'ereditarietà da componenti Swing semplifica l'integrazione grafica e la gestione degli eventi, permettendo riutilizzo di funzionalità comuni.

2.4 Polymorphism

Il polimorfismo è applicato principalmente attraverso:

- Listener e Lambda Expressions: ad esempio in *GestorePannelloRicerca*, dove metodi come *addDocumentListener()* e *addListSelectionListener()* accettano funzioni lambda che implementano interfacce funzionali.
 - Overriding: classi come *PannelloUtente* o *PannelloLinea* ridefiniscono comportamenti specifici di visualizzazione e interazione rispetto a JPanel.
-

2.5 Abstraction

Separazione tra *Model*, *View* e *Controller* in package distinti.

- Le *View* espongono metodi getter per componenti UI
- I *Controller* gestiscono eventi e logica
- I *Model* forniscono solo dati e metodi di accesso

MVC garantisce un livello di astrazione architetturale.

2.6 Design Patterns

- **MVC (Model-View-Controller):** separazione logica dati, interfaccia grafica e logica applicativa
 - **Observer:** eventi Swing e `DocumentListener` notificano automaticamente i cambiamenti
 - **Singleton (parziale):** alcune classi come *GestoreMappa* gestiscono risorse condivise
-

3. Architectural & Project Management Considerations

3.1 Scalability

- Architettura modulare, pannelli e controller autonomi
- Possibile aggiungere funzionalità future senza modificare l'intera base di codice
- Separazione Model/View permette integrazione futura con database o servizi online GTFS-RT

3.2 Maintainability

- Codice documentato con Javadoc e commenti
- Naming convention chiara e coerenza tra i package
- Metodi brevi e coesi, separazione delle responsabilità
- Nuovo sviluppatore può comprendere il flusso leggendo i pacchetti:
org.damose.model → *org.damose.view* → *org.damose.controller* → *org.damose.Main*

3.3 Testability

- Model facilmente testabile con unit test
- Controller testabile isolando la logica

4. External Help and References

Durante lo sviluppo del progetto **Damose**, è stato utilizzato un supporto esterno di tipo intelligente per ottimizzare e approfondire alcune componenti tecniche del sistema. L'assistenza fornita dall'intelligenza artificiale è stata impiegata principalmente come

strumento di ricerca e di supporto alla progettazione, con particolare attenzione a specifiche aree del software:

- **Gestione dei dati in tempo reale (Realtime):** aiuto nella comprensione e nell'integrazione dei feed GTFS Realtime di Roma Mobilità, tramite l'utilizzo della libreria `org.mobilitydata:gtfs-realtime-bindings:0.0.8`.
L'assistenza ha contribuito alla definizione delle classi `GestoreRealTime` e `GestoreMode`, dedicate rispettivamente al download periodico dei dati `.pb` e all'aggiornamento dinamico delle posizioni dei veicoli sulla mappa.
- **Visualizzazione dei percorsi su mappa:** supporto nella gestione grafica delle shape e dei waypoint delle fermate, con l'utilizzo della libreria `org.jxmapviewer:jxmapviewer2:2.8`.
L'intelligenza artificiale ha fornito indicazioni relative al disegno dei percorsi, alla gestione dei painter personalizzati e alla sincronizzazione tra la mappa e i dati del modello.
- **Barra di ricerca in tempo reale:** supporto nella gestione delle barra di ricerca e nell' mostrare i risultati aggiornati in tempo reale in base a quello che si è scritto
- **Ottimizzazione architetturale:** suggerimenti per la strutturazione del progetto secondo il pattern **MVC**, con separazione chiara tra i package `model`, `view` e `controller`.
Inoltre, è stato fornito supporto anche nella documentazione **Javadoc** per migliorare la leggibilità e la manutenibilità del codice.

L'utilizzo dell'assistenza esterna è stato svolto in modo critico e controllato: ogni soluzione proposta è stata analizzata, modificata e adattata al contesto progettuale, mantenendo la piena autonomia decisionale sul design e sul codice finale.

Conclusion

Il progetto Damose rappresenta un esempio completo di architettura MVC applicata a un sistema di gestione di dati GTFS.

Grazie all'uso di incapsulamento rigoroso e separazione dei ruoli, il software è scalabile, modulare e manutenibile.

Le scelte progettuali riflettono una chiara comprensione dei principi OOP e dell'uso corretto dei pattern architetturali.